

Course : MCA Sem II

Subject : Python Programming

(Subject Code: IT21)

Chapt 2: Python Functions, Modules & Packages

Objectives:

1. To learn basic concepts of Functions.
2. To study use of Python Built-in Functions.
3. To understand User defined Functions with its Definition, Calling, Arguments Passing etc.
4. To study Scope of Variables like Global and Local.
5. To learn Module concept with writing and importing Modules.
6. To study Python Built-in Modules like Numeric, Mathematical, Functional Programming Module.
7. To learn Python Packages with its Basic concepts and User defined Packages.

2.1 FUNCTION BASICS SCOPE, NON-LOCAL STATEMENTS

1. A function is a block of organized, reusable code that is used to perform a single,related action/operation. Python has excellent support for functions.
2. A function can be defined as the organized block of reusable code which can be called whenever required. A function is a piece of code that performs a particular task.
3. A function is a block of code which only runs when it is referenced. Python gives us many built-in functions like `print()`,`input()`, `compile()`,etc. but we can also create our own functions called as user-defined

2.1 FUNCTION BASICS SCOPE, NON-LOCAL STATEMENTS

Functions.

1. 2.1.1 User-defined Functions

- Functions in Python programming are self-contained programs that perform some particular tasks. Once, a function is created by the programmer for a specific task, this function can be called anytime to perform that task.

- Python gives us many built-in functions like `print()`, `len()` etc. but we can also create our **own functions**. These functions are called **user-defined functions**.

User defined function are the self-contained block of statements created by according to their requirements..

A user-defined function is a block of related code statements that are organize achieve a single related action or task. A key objective of the concept of the defined function is to encourage modularity and enable reusability of

2.1 FUNCTION BASICS SCOPE, NON-LOCAL STATEMENTS

Functions.

2.1.2 Function Definition:

Function definition is a block where the statements inside the function body written. Functions allow us to define a reusable block of code that can be repeatedly in a program.

Syntax:

```
def function_name(parameters):  
    "function_docstring"  
    function statements  
    return [expression]
```

Chapt 2: Python Functions, Modules & Packages

Defining Function:

1. Function blocks begin with the keyword `def` followed by the function name a parentheses `()`
2. Any input parameters or arguments should be placed within these parentheses can also define parameters inside these parentheses..
3. The first statement of a function can be an optional statement the documentation string of the function or docstring.
4. The code block within every function starts with a colon: and is indented.
5. The statement `return [expression]` exits a function, optionally passing back a expression to the caller. A return statement with no arguments is the same as `return None`.

Chapt 2: Python Functions, Modules & Packages

Advantages of User-defined Functions:

1. User-defined functions help decompose large program into small segments which makes program easy understand, maintain and debug.
2. By using functions, we can avoid rewriting same logic/code again again in program.
3. We can call Python functions any number times in program and from place in program.
4. We can track large Python program easily when it divided into multiple functions.
5. Reusability is the main achievement of Python functions.
6. Functions in Python reduce the overall size of the program

Chapt 2: Python Functions, Modules & Packages

2.1.3 Function Calling

1. The def statement only creates function but does not call After the def has we can call (run) the function by adding parentheses after function name.

Example: For calling function.

```
>>> def square(x):      #function definition
    return x*x
>>> square(4) #function call
```


Chapt 2: Python Functions, Modules & Packages

Concept of Actual and Formal Parameters

1. **Actual Parameters:** The parameters used the function call are called actual parameters. These are the **actual values** that are passed to function. actual parameters may be in the form of **constant values** **variables**. The **data types actual parameters must match** with the corresponding data types formal parameters (variables) in the function definition.

- (i) They are used in the function call.
- (ii) They are actual values that are passed to the function definition through the function call.
- (iii) They can be constant values or variable names (such as local global).

2. **Formal Parameters:** The parameters used in the header of function definition are called formal parameters of the function. These parameters are used to receive values from the calling function.

They are treated as local variable of a function in which they are used in the function header.

Chapt 2: Python Functions, Modules & Packages

Python Programming (MCA Management: Sem. II) Example: For actual and formal parameters.2.4Python Functions, Modules & Packa# formal parameters# actual parameters>>> def cube(x): return x*x*x>>> result = cube (7) >>> print(result)>>> 343

Chapt 2: Python Functions, Modules & Packages

2.1.5 Call by Object Reference .

Most programming languages have a formal mechanism for determining if parameter receives a copy of the argument (call by value) or a reference to argument (call by name or call by reference) but Python uses a mechanism, which known as "Call-by-Object/Call by Object Reference/Call by Sharing".

Example:

```
>>> def increment (n):
        n=n+1
>>> a=3
>>> increment (3)
>>> print(a)
3
```

When we pass a to increment(n), the function has the local variable n referred to the same object. Fig. 2.2 (a): The local variable n referred to the same object. When control comes to $n = n+1$ as integer is immutable, by definition we are not able to modify the object's value to 4 in place: we must create a new object with the value 4. We may visualize it like below: Fig. 2.2 (b): Create a new object with the value 4. All

Chapt 2: Python Functions, Modules & Packages

Python Functions, Modules & Packages. By assigning the return value of the function to a, we have reassigned a to refer to the new object with the value 4 created in the function. Note, the object a initially referred to never change; it is still 3 but by having a point to a new object created by the function, we are able to "modify" the value of a. Passing mutable objects: The same increment() function generates a different result when we passing a mutable object: Here L is a list which is mutable..>>> def increment (n): n.append([4])>>> L=[1,2,3] >>> increment (L)>>> print(L) [1, 2, 3, [4]] Here, the statement L= [1, 2, 3] makes a variable L(box) that points towards the object [1, 2, 3]. On the function being called, a new box n is created. The contents of n are the same as the contents of box L. Both the boxes contain the same object. That is, both the variables point to the same object in memory. Hence, any change to the object pointed at by n will also be reflected by the object pointed at by L.

Fig. 2.2 (d): Mutable Objects

Hence the output of the above program will be:[1, 2, 3, 4]

Chapt 2: Python Functions, Modules & Packages

Modules & Packag Python Programming (MCA Management: Sem. II)2.1.7
Recursive Function Recursion is a way of programming or coding a problem, in which a function cal itself one or more times in its body. A function is said to be a recursive if it calls itself. For example, let's say we def fact(n): if n = 0: return 1 function abc() and in the body of abc() there is a call to the abc(). Example: For recursive function. print(fact(0)) print(fact(6)) else: return n fact (n-1) print(fact (4)) Output: 124720

Chapt 2: Python Functions, Modules & Packages

The factorial of 4 (denoted as 4!) is $1 \cdot 2 \cdot 3 \cdot 4 = 24$. Each function call multiplies the number with the factorial of number 1 until the number is equal to one.

fact(4) # 1st call with 4 # 2nd call with 3 # 3rd call with 2 # 4th call with 1 # returns from 4th call as number 1 # returns from 3rd call # returns from 2nd call # returns from 1st call

Our recursion ends when the number reduces to 1. This is called the base condition. Every recursive function must have a base condition that stops the recursion or else the function calls itself infinitely.

Chapt 2: Python Functions, Modules & Packages

Advantages of Recursion:

- 1 Recursive functions make the code look clean and elegant.
2. A complex task can be broken down into simpler sub-problems using recursion.
3. Sequence generation is easier with recursion than using some nested iteration.

Disadvantages of Recursion:

1. Sometimes the logic behind recursion is hard to follow through.
2. Recursive calls are expensive (inefficient) as they take up a lot of memory and time.
3. Recursive functions are hard to debug.
4. It consumes more storage space because the recursive calls along with variables are stored on the stack.
5. It is not more efficient in terms of speed and execution time.

Chapt 2: Python Functions, Modules & Packages

2.1.8 Scope of Variable. All variables in a program may not be accessible at all locations in that program. This depends on where we have declared a variable. The scope of a variable determines the portion of the program where you can access a particular variable/identifier. The availability/accessibility of a variable in different parts of a program is referred to as its scope. There are two basic scopes of variables in Python: 1. Global Variables: Global variables can be accessed throughout (outside) the program body by all functions. 2. Local Variables: Local variables can be accessed only inside the function in which they are declared. Code Block 1 Code Block 2 Code Block 3X

Chapt 2: Python Functions, Modules & Packages

Fig. 2.3: Scopes of variables. Fig.2.3 shows, global variable (x) can be reached and modified anywhere in the code, local variable (z) exists only in block 3. Example: For scope of variables.

```
g=10 #global variable
def show():
    z=20 #local variable
    print("local variable-",z)
print("Global variable=",g)
show()
```

Output: local variable- 20 Global variable 10

Chapt 2: Python Functions, Modules & Packages

[8:19 am, 09/06/2022] Prashant Wadkar: 2.1: Difference Global Variable Local Variable Global Variable variables function. Global variables any function. Accessed only by the statements, inside declared. 2. Accessed any statement the variables function. Global variables alive till the end of local variable destroyed when the 4. Global variable destroyed when the control of program exit out block Nonlocal statements Nonlocal variables used in nested functions whose local scope not defined. means the variable can neither the local the global scope. "nonlocal keyword variables. Example: func(): 1020 def innerfunc(): binding nonlocal 100200 will update #b will not update, because will be considered a local variable calling inner function innerfunc() #printing the value a and b print("a", a) print("b: b). #main code calling the function i.e. outerfunc() outerfunc() Output: a: 10020